

UNIVERSIDADE FEDERAL DE SERGIPE

PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

CLARIANE MARIA OLIVEIRA LISBOA

IRIS AZENATE FERREIRA

JOSÉ NILTON DOS SANTOS OLIVEIRA

LEONARDO SOUZA SILVA

THIAGO DE JESUS DOS SANTOS

VICTÓRIA CARDOSO DOS SANTOS

RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED GENERATION (RAG): SELEÇÃO DE CONTEXTO COM PROGRAMAÇÃO DINÂMICA

São Cristóvão, Sergipe
2026

CLARIANE MARIA OLIVEIRA LISBOA

IRIS AZENATE FERREIRA

JOSÉ NILTON DOS SANTOS OLIVEIRA

LEONARDO SOUZA SILVA

THIAGO DE JESUS DOS SANTOS

VICTÓRIA CARDOSO DOS SANTOS

RECUPERAÇÃO EFICIENTE DE CONTEXTO EM ARTIGOS CIENTÍFICOS SOBRE MODELOS DE LINGUAGEM, RECUPERAÇÃO DE INFORMAÇÃO E RETRIEVAL-AUGMENTED GENERATION (RAG): SELEÇÃO DE CONTEXTO COM PROGRAMAÇÃO DINÂMICA

Trabalho apresentado à disciplina de Projeto e Análise de Algoritmos da Universidade Federal de Sergipe, como requisito parcial para obtenção de nota na referida disciplina.

Professor: Prof. Glauco Carneiro

São Cristóvão, Sergipe
2026

SUMÁRIO

1 INTRODUÇÃO **4**
2 METODOLOGIA **4**
2.1. Métodos de Seleção 4
2.2. Ranking Simples 5
2.3. Método Guloso 5
2.4. Programação Dinâmica 5
2.4.1. Implementação 6
2.4.2. Reconstrução da Solução 6
2.4.3. Demonstração da Programação Dinâmica 7
2.5. Abordagem Exaustiva, Validação e Corretude 8
3. RESULTADOS **8**
3.1. Caracterização dos custos e do orçamento de contexto 8
4 . ANÁLISE ASSINTÓTICA **11**
5. APLICAÇÃO NO RAG E LIMITAÇÕES **12**
6 . CONCLUSÃO **12**
7 DECLARAÇÃO DE USO DE IA GENERATIVA **13**
8 TABELA DE CONTRIBUIÇÃO INDIVIDUAL **14**
9 URL DO VÍDEO E REPOSITÓRIO **15**
REFERÊNCIAS BIBLIOGRÁFICAS **15**

1 INTRODUÇÃO

A Geração Aumentada por Recuperação (Retrieval-Augmented Generation – RAG) combina modelos de linguagem com mecanismos de recuperação de informações externas para complementar o contexto utilizado na geração de respostas (GAO et al., 2023). Entretanto, a quantidade de documentos recuperados pode ultrapassar o orçamento disponível de tokens, tornando necessária a seleção dos conteúdos que serão enviados ao modelo.

Nesse contexto, a escolha dos artigos pode ser tratada como um problema da mochila 0/1. Cada artigo tem uma pontuação lexical e ocupa uma quantidade de tokens. O objetivo é escolher o conjunto com a maior soma de pontuações sem ultrapassar o limite disponível. A Programação Dinâmica permite encontrar essa combinação (MIT OPENCOURSEWARE, 2011; CP-ALGORITHMS, s.d.).

Este trabalho avalia a Programação Dinâmica na seleção de artigos científicos da ACL Anthology e compara seus resultados com o Ranking AV1 e o método Guloso. A comparação considera o score lexical, a precisão semântica, o tempo de execução e o consumo de memória.

2 METODOLOGIA

O experimento deu continuidade à AV1, na qual a coleta foi configurada para até 18.000 artigos da ACL Anthology. Foram utilizados os primeiros 500 e 1.000 artigos, com seleção dos 20 ou 50 candidatos mais bem pontuados. Os custos foram calculados com o tokenizador “cl100k_base”, e os métodos Ranking AV1, Guloso e Programação Dinâmica foram comparados nos orçamentos de 708 e 1.416 tokens, considerando as consultas Q1, Q3 e Q5. Cada configuração foi executada três vezes, totalizando 216 medições. Foi executada no Google Colab, sistema: Linux-6.6.159+-x86_64-with-glibc2.3, python: 3.13.18. Foram utilizadas as bibliotecas NumPy 2.4.6, pandas 2.2.3, NLTK 3.10.3, sentence-transformers 5.7.0 e scikit-learn 1.6.1.

2.1. Métodos de Seleção

A seleção funciona como uma mochila: entre N artigos candidatos, cada um possui uma pontuação lexical (v_i) e um custo em tokens (c_i). O limite W representa a quantidade de tokens disponível. Os métodos tentam escolher artigos com boa pontuação sem passar desse limite. Foram comparados o Ranking AV1, o Guloso e a Programação Dinâmica; a Busca Exaustiva foi usada apenas para conferir resultados em casos pequenos.

O custo de cada artigo foi definido como a quantidade de tokens de seu título e resumo, calculada com a biblioteca tiktoken (OPENAI, 2022), utilizando a codificação "cl100k_base". Essa codificação emprega o algoritmo *Byte Pair Encoding* (BPE), cuja aplicação à segmentação de textos em subpalavras foi proposta por Sennrich, Haddow e Birch (2016). Na variante utilizada pelo tiktoken, o BPE opera sobre sequências de bytes, combinando padrões frequentes para formar unidades de codificação. Assim, um token pode representar uma palavra inteira, parte de uma palavra ou um símbolo, não correspondendo necessariamente a uma palavra completa. A quantidade de tokens obtida foi utilizada como o custo de cada artigo no problema da mochila 0/1.

Para avaliar os métodos, foram usadas três consultas da AV1, apresentadas a seguir. Cada consulta foi aplicada aos dois tamanhos de corpus; os resultados das tabelas consideram as médias de Q1, Q3 e Q5.

Quadro 1 – Consultas utilizadas no experimento

Consulta	**Pergunta**
Q1	What information retrieval methods are used in RAG systems?
Q3	What semantic search techniques are used to retrieve context for language models?
Q5	What metrics are used to evaluate Retrieval-Augmented Generation systems?

Fonte: adaptado do Relatório 1 (2026).

2.2. Pontuação lexical e avaliação dos artigos

Para entender os resultados, é importante separar duas coisas: a pontuação que os algoritmos procuram aumentar e a relevância usada para verificar se os artigos escolhidos realmente tratam da consulta.

Pontuação lexical: Na AV1, cada artigo foi representado pelo título e pelo resumo. Os textos foram normalizados e os termos da consulta passaram por tokenização, remoção de palavras comuns (stopwords) e stemming. A pontuação conta quantas vezes os termos da consulta aparecem no artigo. Assim, um artigo que contém mais desses termos recebe um valor maior. Para cada consulta, os 20 ou 50 candidatos são os artigos com as maiores pontuações.

Score lexical:. É a soma das pontuações dos artigos que couberam no orçamento. Por exemplo, se forem escolhidos dois artigos com valores 12 e 8, o score lexical será 20. Esse é o número que a Programação Dinâmica busca maximizar. Como os artigos e suas pontuações não mudam entre repetições, o score também não muda.

Relevância semântica: Para avaliar a qualidade da seleção, a AV1 criou um gabarito diferente da pontuação lexical. O modelo all-MiniLM-L6-v2 transforma a consulta e os títulos e resumos dos artigos em representações numéricas (embeddings). A similaridade de cosseno indica quais textos estão mais próximos do assunto pesquisado. Os 15 artigos mais próximos em todo o corpus formam o gabarito de cada consulta. Esse gabarito é usado na avaliação, não na escolha dos artigos.

Precisão: Mostra a proporção dos artigos selecionados que também aparecem no gabarito. Se o método escolhe três artigos e dois estão no gabarito, sua precisão é 2 ÷ 3, ou aproximadamente 0,667.

Precision@5 (P@5): Conta quantos dos cinco primeiros artigos considerados na avaliação aparecem no gabarito e divide esse número por 5. Quando a seleção tem menos de cinco artigos, esse limite afeta o resultado: com três artigos selecionados, a P@5 não pode passar de 0,600. A ordem usada para identificar os cinco primeiros precisa ser conferida no notebook da AV2.

Jaccard, tempo e memória: O índice de Jaccard mostra a sobreposição entre os textos selecionados: valores menores indicam menos repetição de termos. O tempo mede

quanto a seleção demora, em milissegundos (ms), e a memória corresponde ao pico registrado pelo tracemalloc, em megabytes (MB). Os números das tabelas são médias das consultas Q1, Q3 e Q5; tempo e memória também consideram as três repetições de execução.

2.3. Ranking Simples

O Ranking AV1 foi mantido como método de comparação (baseline). Ele coloca os artigos em ordem decrescente de pontuação lexical e, nessa ordem, inclui aqueles que ainda cabem no limite de tokens. Como não considera o custo na ordenação, pode deixar de encontrar um conjunto com score lexical maior. O código está no Quadro A.1, no Apêndice A.

2.4. Método Guloso

O método Guloso é parecido com o Ranking AV1, mas dá prioridade aos artigos que oferecem mais pontuação lexical por token. Ele percorre essa lista e inclui os artigos que cabem no espaço restante. Em caso de empate, considera primeiro a maior pontuação e depois o identificador. Como não volta atrás para trocar artigos já escolhidos, não garante a melhor soma possível. A diferença de ordenação está indicada no Apêndice A.

2.5. Programação Dinâmica

A Programação Dinâmica compara as opções de incluir ou não cada artigo e guarda os melhores resultados parciais em uma tabela. Assim, evita refazer os mesmos cálculos e consegue encontrar a maior soma de pontuações possível dentro do orçamento de tokens (CORMEN et al., 2022).

Na tabela, T[i][w] representa a melhor soma de pontuações possível ao considerar os i primeiros artigos e um limite de w tokens. O resultado final fica em T[N][W].

O Quadro 1 mostra como a tabela é preenchida.

Quadro 1 – Recorrência da programação dinâmica para a mochila 0/1

Caso base:	$T[0][w] = 0,$	para todo w em $\{0, \ldots, W\}$
Se $c_i > w$:	$T[i][w] = T[i-1][w]$	(o artigo i não cabe)
Se $c_i \leq w$:	$T[i][w] = \max($ $T[i-1][w],$	(não pegar o artigo i)
	$\quad T[i-1][w - c_i] + v_i)$	(pegar o artigo i)

Fonte: elaboração própria, com base em Cormen et al. (2022).

As linhas indicam quantos artigos já foram considerados; as colunas mostram o limite de tokens. A linha e a coluna iniciais representam os casos em que ainda não é possível selecionar artigos.

2.5.1. Implementação

A implementação usa duas matrizes NumPy. A matriz tabela guarda a maior pontuação encontrada para cada limite de tokens. A matriz escolhido registra se valeu a pena incluir o artigo naquela etapa. Com isso, ao final, é possível descobrir quais artigos formam a seleção.

Para cada artigo, o algoritmo começa mantendo o melhor resultado anterior. Artigos que ultrapassam sozinhos o orçamento ou têm pontuação zero são ignorados. Nos demais casos, o código compara incluir ou não o artigo e atualiza as matrizes. O uso de operações vetorizadas do NumPy evita percorrer cada orçamento em um laço separado. A implementação completa está no Quadro A.2, no Apêndice A.

2.5.2. Reconstrução da Solução

Depois de preencher a tabela, o algoritmo percorre as decisões de trás para frente para recuperar os artigos escolhidos, em O(N). O código também verifica duas condições: o total de tokens não pode ultrapassar W e o score da seleção precisa ser igual ao melhor valor calculado. A reconstrução aparece no Quadro A.2, no Apêndice A.

2.5.3. Demonstração da Programação Dinâmica

Considere uma mochila com orçamento W = 5 tokens e os artigos descritos na Tabela 1.

Tabela 1 - Instância de exemplo

Artigo	Índice (*i*)	Custo (tokens)	Valor
A	1	2	3
B	2	3	4
C	3	1	2

Fonte: elaboração própria (2026).

O exemplo tem três artigos e um orçamento de cinco tokens. A linha inicial da matriz representa a situação em que nenhum artigo foi considerado. O artigo A custa 2 tokens e vale 3 pontos; o B custa 3 e vale 4; o C custa 1 e vale 2. As Tabelas 2 e 3 mostram, respectivamente, a melhor pontuação para cada limite e as decisões tomadas em cada etapa.

Tabela 2 - Matriz tabela (Resultados Máximos)

	w = 0	*w* = 1	*w* = 2	*w* = 3	*w* = 4	*w* = 5
i = 0 (nenhum)	0	0	0	0	0	0
i = 1 (artigo A)	0	0	3	3	3	3
i = 2 (artigo B)	0	0	3	4	4	7
i = 3 (artigo C)	0	2	3	5	6	7

Fonte: elaboração própria (2026).

Tabela 3 - Matriz escolhido (histórico de decisão)

	w = 0	*w* = 1	*w* = 2	*w* = 3	*w* = 4	*w* = 5
i = 1 (artigo A)	False	False	True	True	True	True
i = 2 (artigo B)	False	False	False	True	True	True
i = 3 (artigo C)	False	True	False	True	True	False

Fonte: elaboração própria (2026).

O melhor resultado é escolher A e B: juntos, eles custam 5 tokens e somam 7 pontos. O artigo C não melhora essa combinação.

2.6. Abordagem Exaustiva, Validação e Corretude

Para verificar a Programação Dinâmica, foi usada uma Busca Exaustiva, que testa todas as combinações possíveis e identifica a melhor delas. Como esse processo fica caro quando há muitos artigos, foi empregado apenas em instâncias pequenas. O código está no Quadro A.3, no Apêndice A.

A validação incluiu testes de situações específicas e uma comparação da Programação Dinâmica com a Busca Exaustiva em 30 instâncias aleatórias. Os 12 testes informados foram aprovados. O Quadro A.4, no Apêndice A, apresenta os 11 casos dirigidos; o teste diferencial é descrito no texto e tem seu código disponível no repositório do grupo.

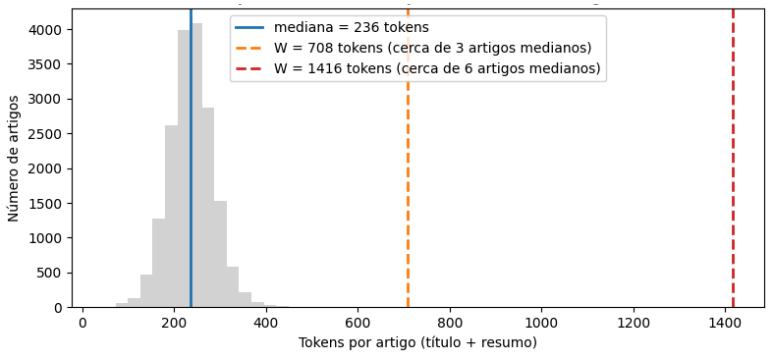
A justificativa da corretude é simples: a cada artigo, o algoritmo compara o resultado de incluí-lo com o de deixá-lo de fora e mantém a melhor opção possível. Os testes com a Busca Exaustiva reforçam essa verificação, embora não substituam a justificativa teórica. As entradas também são verificadas para impedir custos inválidos, orçamentos negativos e identificadores repetidos.

3. RESULTADOS

3.1. Caracterização dos custos e do orçamento de contexto

A distribuição dos custos dos artigos foi analisada para fundamentar a definição dos orçamentos experimentais. Na Figura 1, observa-se que a mediana dos 18.000 artigos foi de 236 tokens por artigo, utilizada como referência para estabelecer dois limites: 708 e 1.416 tokens, correspondentes a três e seis vezes a mediana, respectivamente.

Figura 1 — Posicionamento dos orçamentos experimentais em relação à distribuição dos custos dos artigos.



Fonte: elaboração própria (2026).

A Tabela 4 compara os métodos em desempenho e custo computacional. O campo Score representa a soma das pontuações lexicais dos artigos escolhidos; Tokens é a soma dos custos; Tempo e Mem. mostram, respectivamente, o tempo em milissegundos e o pico de memória em megabytes. Precisão e P@5 indicam a presença de artigos do gabarito semântico na seleção; Jaccard mostra a sobreposição entre os selecionados. Nesse recorte, a Programação Dinâmica obteve o maior score lexical médio (74,67), mas usou mais tempo (entre 1,2 e 1,3 ms) e memória (cerca de 0,32 MB) do que o Ranking e o Guloso.

Tabela 4 – Comparação do desempenho e do custo computacional dos métodos de seleção de contexto. Valores médios para N = 50 candidatos e W = 708 tokens, considerando Q1, Q3 e Q5, com três repetições.

Corpus	Método	Score	Tempo (ms)	Mem. (MB)	Tokens	Precisão	Jaccard
500	Ranking	60,33	0,116	0,0031	675,00	0,722	0,194
500	Guloso	72,67	0,122	0,0031	669,00	0,500	0,172
500	DP	74,67	0,69	0,3216	699,33	0,639	0,187
1.000	Ranking	60,33	0,129	0,0031	675,00	0,722	0,194
1.000	Guloso	72,00	0,126	0,0031	661,00	0,639	0,181
1.000	DP	74,67	0,697	0,3213	698,33	0,639	0,197

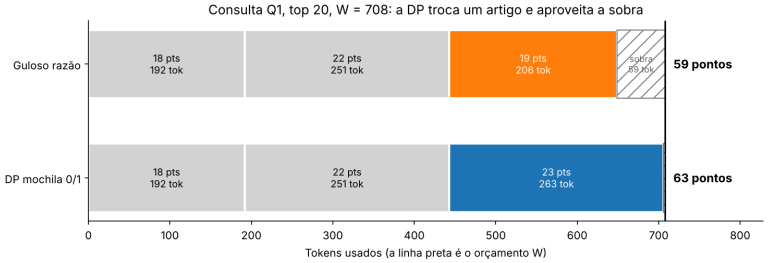
Fonte: elaboração própria (2026).

A Programação Dinâmica alcançou scores lexicais iguais ou maiores que os métodos de comparação. No cenário de 500 artigos, N = 20 candidatos e W = 708 tokens, que não aparece na Tabela 4, as médias foram 73,00 (DP), 70,00 (Guloso) e 60,33 (Ranking AV1). Ao ampliar para N = 50, o score da DP passou a 74,67. Considerando os 24 cenários diferentes do experimento, a DP superou o Guloso em 16 e empatou nos demais. As três repetições de cada cenário não representam novos resultados de score.

O Ranking AV1 apresentou a maior precisão semântica no recorte da Tabela 4 (0,722), mesmo tendo score lexical menor. A Programação Dinâmica chegou a 0,639. Isso mostra que encontrar mais ocorrências dos termos da consulta não significa, necessariamente, selecionar mais artigos próximos do assunto segundo o gabarito semântico. O Guloso teve a menor sobreposição medida por Jaccard (0,172 a 0,181).

A Figura 2 apresenta um exemplo da consulta Q1, com 20 candidatos e orçamento de 708 tokens. Nesse cenário, o método Guloso selecionou três artigos, totalizando 59 pontos e utilizando 649 tokens. A Programação Dinâmica, por sua vez, encontrou uma combinação com 63 pontos e 706 tokens. O exemplo demonstra que a DP conseguiu uma pontuação superior ao substituir um dos artigos selecionados pelo Guloso, sem ultrapassar o orçamento disponível.

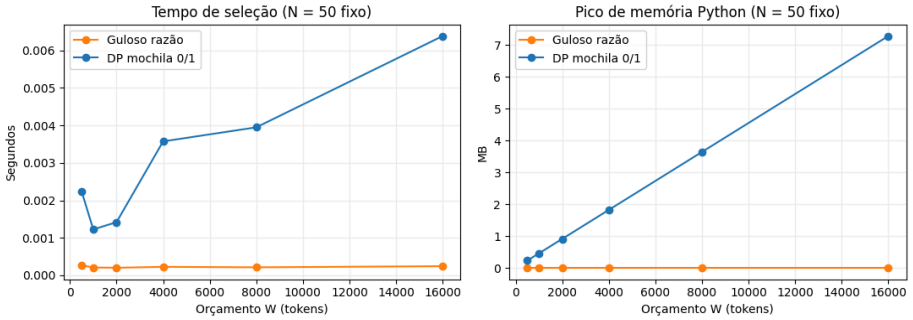
Figura 2 – Exemplo de seleção de contexto na consulta Q1, com top 20 candidatos e W = 708 tokens, comparando o método Guloso por razão e a Programação Dinâmica.



Fonte: elaboração própria (2026).

A Figura 3 apresenta o impacto do orçamento de tokens (W) sobre o tempo de seleção e o consumo de memória dos métodos Guloso e Programação Dinâmica, mantendo N = 50 candidatos. O Guloso apresentou tempo de execução relativamente estável e baixo consumo de memória à medida que o orçamento aumentou. Em contrapartida, a Programação Dinâmica apresentou maior custo computacional, com crescimento do tempo de execução e pico de memória de aproximadamente 7,3 MB para W = 16.000 tokens.

Figura 3 – Tempo de seleção e consumo de memória em função do orçamento de tokens para $N = 50$ candidatos.



Fonte: elaboração própria (2026).

A Figura 3 mostra que a Programação Dinâmica apresentou maior tempo de execução e consumo de memória em comparação com o Ranking AV1 e o Guloso. Esse comportamento está relacionado à necessidade de armazenar os resultados intermediários para encontrar a melhor combinação de artigos. Assim, embora a Programação Dinâmica obtenha maior pontuação lexical, isso ocorre com um custo computacional mais elevado.

4. ANÁLISE ASSINTÓTICA

A análise assintótica considera o tempo de execução e o consumo de memória dos algoritmos utilizados na seleção de contexto. A Tabela 5 apresenta as complexidades do Ranking AV1, Guloso, Programação Dinâmica e Busca Exaustiva, considerando N como o número de artigos candidatos e W como o orçamento de tokens disponível.

Tabela 5 – Análise assintótica dos algoritmos de seleção de contexto.

Algoritmo	Melhor caso	Pior caso	Complexidade espacial
Ranking AV1	Θ(N)	Θ(N log N)	Θ(N)
Guloso	Θ(N)	Θ(N log N)	Θ(N)
Programação Dinâmica	Θ(N × W)	Θ(N × W)	Θ(N × W)
Busca Exaustiva	Θ(N × 2^N)	Θ(N × 2^N)	Θ(N)

Fonte: Elaborado pelos autores (2026), com base em Erickson (2019) e CP-Algorithms (2026).

O Ranking AV1 e o Guloso usam $\Theta(N \log N)$ de tempo, no pior caso, por causa da ordenação, seguida de uma única passagem pelos candidatos, e $\Theta(N)$ de espaço para guardar a lista ordenada. Como fazem escolhas em sequência, não garantem a melhor combinação possível (Erickson, 2019).

A Programação Dinâmica apresenta complexidade temporal e espacial $\Theta(N \times W)$, pois preenche uma tabela de $(N + 1) \times (W + 1)$ células, e cada célula é calculada em tempo constante a partir da linha anterior (CP-Algorithms, 2026; MIT OpenCourseWare, 2011). A reconstrução dos artigos selecionados percorre a tabela uma vez por item, com custo adicional $\Theta(N)$, que não altera a ordem de complexidade total. Já a Busca Exaustiva possui custo temporal exponencial, $\Theta(N \times 2^N)$, pois avalia os 2^N subconjuntos possíveis e soma os custos e valores de cada um em tempo $O(N)$. Por esse motivo, ela é utilizada apenas para validar a solução da Programação Dinâmica em instâncias pequenas, com no máximo 20 itens.

Conforme apresentado na Figura 3, o aumento do orçamento de tokens provoca um crescimento mais acentuado no tempo de execução e no consumo de memória da Programação Dinâmica, enquanto o Ranking AV1 e o Guloso apresentam custos computacionais menores. Para $N = 50$, os picos de memória da Programação Dinâmica passaram de aproximadamente 0,32 MB, com $W = 708$, para 7,3 MB, com $W = 16.000$. Esse comportamento ocorre porque o método precisa armazenar resultados intermediários em matrizes cujo tamanho aumenta conforme o orçamento disponível.

O modelo RAM considera apenas a etapa de seleção. Ficam fora dele a tokenização e a construção do índice, executadas na preparação dos dados, a geração dos embeddings do gabarito, utilizada apenas na avaliação, o acesso a disco e, em uma aplicação RAG real, a geração de respostas pelo LLM, cujo custo não foi mensurado neste experimento.

5. APLICAÇÃO NO RAG E LIMITAÇÕES

Em sistemas RAG, os documentos recuperados são utilizados como contexto para auxiliar um modelo de linguagem na geração de respostas. Entretanto, existe um limite de tokens disponíveis, o que torna necessário escolher quais documentos serão incluídos. Nesse cenário, a Programação Dinâmica pode contribuir ao buscar a combinação de artigos com maior pontuação lexical, respeitando o orçamento estabelecido.

No entanto, o método apresenta limitações. A Programação Dinâmica exige mais memória e tempo de execução que as estratégias de Ranking e Guloso avaliadas, especialmente conforme aumentam os candidatos e o orçamento. Além disso, uma maior pontuação lexical não significa necessariamente maior relevância semântica, pois a pontuação considera a ocorrência dos termos da consulta.

Outra limitação está no gabarito semântico, construído automaticamente por um único modelo de embeddings, sem validação humana. O experimento também não avaliou as respostas geradas por um modelo de linguagem. Portanto, embora a Programação Dinâmica consiga maximizar a pontuação lexical dentro do orçamento, não é possível afirmar que sua utilização resulte em respostas melhores em sistemas RAG.

6. CONCLUSÃO

Os resultados mostram que a Programação Dinâmica encontrou a melhor combinação possível para a soma das pontuações lexicais, respeitando o orçamento de tokens. O Guloso foi mais rápido e consumiu menos memória, mas não garante a solução de maior score lexical.

Já o Ranking AV1 teve precisão semântica maior (0,722) do que a Programação Dinâmica (0,639) no recorte apresentado na Tabela 4. Portanto, aumentar o score lexical não significa necessariamente escolher artigos semanticamente mais próximos da consulta. A escolha do método depende do equilíbrio entre pontuação, precisão e custo computacional.

7 DECLARAÇÃO DE USO DE IA GENERATIVA

Em conformidade com as orientações de transparência no uso de Inteligência Artificial Generativa, os autores declaram a utilização da ferramenta ChatGPT, desenvolvida pela OpenAI, como recurso de apoio à revisão textual, à organização das informações e à conferência de consistência técnica do presente trabalho. Durante a revisão do relatório, foi utilizado o modelo GPT-6.

A ferramenta foi empregada de forma complementar, auxiliando na identificação de possíveis inconsistências, na melhoria da clareza da linguagem técnica e na organização da apresentação dos métodos e resultados. As sugestões foram avaliadas criticamente pelos integrantes, considerando as implementações e os dados obtidos no experimento.

Quadro 2 – Registro de utilização da IA generativa.

Prompt utilizado (resumido)	**Tratamento da sugestão**
“Revise a clareza e a organização da descrição dos métodos de seleção.”	**Aproveitada parcialmente:** ajustes na redação e na organização das subseções, preservando a metodologia apresentada pelo grupo.
“Verifique se a descrição da Programação Dinâmica está coerente com o código implementado.”	**Aproveitada após conferência:** revisão da explicação das matrizes, do preenchimento e da reconstrução da solução, com base no notebook.
“Confira a consistência entre as métricas, as tabelas e a discussão dos resultados.”	**Corrigida após verificação:** ajustes pontuais na apresentação das medições e na interpretação dos resultados experimentais.
“Sugira formas de reduzir o relatório sem comprometer as informações essenciais.”	**Parcialmente rejeitada:** a retirada de figuras não foi adotada; foram mantidos os gráficos e reduzidos apenas trechos repetitivos.
“Verifique se a implementação da Programação Dinâmica para o problema da mochila 0/1 está correta, considerando o preenchimento da tabela, a reconstrução da solução e o respeito ao orçamento de tokens.”	**Aproveitada como apoio à revisão**: as observações foram conferidas no código e a corretude foi avaliada por meio de testes automatizados, incluindo a comparação com a Busca Exaustiva em 30 instâncias aleatórias.

Fonte: Elaborado pelos autores (2026).

Os autores permanecem responsáveis pelas decisões metodológicas, pela validação das implementações, pela interpretação dos resultados e pela integridade do conteúdo apresentado. A ferramenta de IA generativa não foi incluída como autora do trabalho.

8 TABELA DE CONTRIBUIÇÃO INDIVIDUAL

Quadro 6 - Contribuição individual dos integrantes.

Integrante	Responsabilidade principal	Contribuições específicas
Clariane Maria Oliveira Lisboa	Modelagem do problema da mochila 0/1	Definição dos orçamentos de tokens, implementação da validação das entradas e padronização do formato de saída dos algoritmos. Edição do vídeo.
Iris Azenate Ferreira	Preparação do corpus e configuração do experimento	Configuração do ambiente, obtenção e preparação dos artigos da ACL Anthology, contagem de tokens e análise da distribuição dos custos.
José Nilton dos Santos Oliveira	Implementação dos métodos de referência	Reutilização da busca e do índice invertido da AV1, geração dos candidatos e implementação do Ranking AV1 e do método Guloso.
Leonardo Souza Silva	Implementação e validação da programação dinâmica	Desenvolvimento da programação dinâmica e da busca exaustiva, implementação dos testes funcionais e de integridade e análise de corretude.
Thiago de Jesus dos Santos	Planejamento experimental e análise dos resultados	Definição e execução das configurações experimentais, coleta das métricas de desempenho e relevância, elaboração das tabelas e gráficos comparativos.
Victória Cardoso dos Santos	Análise assintótica e aplicação no RAG	Análise das complexidades temporal e espacial, avaliação do comportamento pseudopolinomial, montagem do contexto para RAG, discussão das limitações e registro de informações de reprodutibilidade.

Fonte: Elaborado pelos autores.

Além das responsabilidades descritas, os integrantes também colaboraram na revisão geral do trabalho, na preparação da apresentação e na gravação do vídeo. A organização e a disponibilização dos arquivos no repositório ficaram sob responsabilidade de Leonardo Souza Silva.

9 URL DO VÍDEO E REPOSITÓRIO

Repositório Github:
https://github.com/Leonardodcomp/PAA_UFS_2026_2_Ferreira_Iris_Ferreira_Jose_Lisboa_Clariane_Santos_Thiago_Santos_Victoria_Silva_Leo/tree/main/AV02

URL do vídeo:
https://drive.google.com/file/d/1MBIx6AbKiQ3P_lAueVdiaLOKJwhNf9he/view

REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIATION FOR COMPUTATIONAL LINGUISTICS. **ACL Anthology**. [S. l.]: Association for Computational Linguistics, [s. d.]. Disponível em: https://aclanthology.org/. Acesso em: 23 set. 2026.

CP-ALGORITHMS. **Knapsack Problem.** [S. l.], [s. d.]. Disponível em: https://cp-algorithms.com/dynamic_programming/knapsack.html.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms**. 4. ed. Cambridge, MA: MIT Press, 2022.

ERICKSON, Jeff. **Algorithms**. Urbana: [s.n.], 2019. Disponível em: https://jeffe.cs.illinois.edu/teaching/algorithms/. Acesso em: 15 set. 2026.

GAO, Yunfan et al. **Retrieval-Augmented Generation for Large Language Models: A Surve**y. arXiv, 2023. Disponível em: https://arxiv.org/abs/2312.10997.

MIT OPENCOURSEWARE. **Dynamic Programming: Knapsack Problem**. Cambridge: Massachusetts Institute of Technology, 2011. Disponível em: https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/.

SENTENCE TRANSFORMERS. **all-MiniLM-L6-v2.** [S.l.]: Hugging Face, [2021?]. Disponível em: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Acesso em: 20 set. 2026.

APÊNDICE A – IMPLEMENTAÇÕES DOS ALGORITMOS

Quadro A.1 – Implementação do Ranking AV1 (base também para o Guloso).

```
def ranking_av1(itens, W):
    """Baseline: tenta os artigos com maiores pontuações primeiro."""
    validar_entrada(itens, W)
    ordem = sorted(itens, key=lambda a: (-a['value'], a['id']))
    escolhidos = []
    espaco_livre = W
    for artigo in ordem:
        if artigo['value'] > 0 and artigo['cost'] <= espaco_livre:
            escolhidos.append(artigo)
            espaco_livre -= artigo['cost']
    return resposta(escolhidos)
```

Fonte: elaboração própria (2026). No Guloso, apenas a ordenação muda para: key=lambda a: (-a['value']/a['cost'], -a['value'], a['id']).

Quadro A.2 – Implementação da Programação Dinâmica (mochila 0/1).

```
def mochila_dp(itens, W):
    """Programação dinâmica: encontra a melhor soma de pontuações."""
    validar_entrada(itens, W)
    quantidade = len(itens)

    # Tabela de valores e tabela para lembrar quais artigos foram
escolhidos.
    tabela = np.zeros((quantidade + 1, W + 1))
    escolhido = np.zeros((quantidade + 1, W + 1), dtype=bool)

    for i, artigo in enumerate(itens, start=1):
        custo = int(artigo['cost'])
        pontos = float(artigo['value'])

        # Começa com a opção de NÃO pegar o artigo atual.
        tabela[i] = tabela[i - 1]
        if custo > W or pontos == 0:
            continue
```

```
        # Se pegar o artigo, o valor é o anterior + pontos dele.
        # O fatiamento faz a comparação para todos os orçamentos de uma
vez.
        valor_se_pegar = tabela[i - 1, :W + 1 - custo] + pontos
        valor_sem_pegar = tabela[i, custo:]
        vale_a_pena = valor_se_pegar > valor_sem_pegar + 1e-10

        tabela[i, custo:] = np.where(vale_a_pena, valor_se_pegar,
valor_sem_pegar)
        escolhido[i, custo:] = vale_a_pena

    # Caminha do final da tabela até o início para descobrir os IDs.
    espaco = W
    artigos_escolhidos = []
    for i in range(quantidade, 0, -1):
        if escolhido[i, espaco]:
            artigo = itens[i - 1]
            artigos_escolhidos.append(artigo)
            espaco -= int(artigo['cost'])

    artigos_escolhidos.reverse()
    resultado = resposta(artigos_escolhidos)
    assert resultado['tokens'] <= W
    assert math.isclose(
        resultado['valor'],
        tabela[quantidade, W],
        abs_tol=1e-7
    )
    return resultado
```

Fonte: elaboração própria (2026).

Quadro A.3 – Implementação da Busca Exaustiva.

```
def exaustiva(itens, W):
    """Confere todas as combinações (somente para testes pequenos)."""
    validar_entrada(itens, W)
    if len(itens) > 20:
        raise ValueError('Use no máximo 20 artigos na busca exaustiva')

    melhor = resposta([])
    for mascara in range(2 ** len(itens)):
        escolhidos = [a for i, a in enumerate(itens) if mascara & (1 << i)]
        atual = resposta(escolhidos)
```

```
        if atual['tokens'] <= W and atual['valor'] > melhor['valor'] +
1e-10:
            melhor = atual
    return melhor
```

Fonte: elaboração própria (2026).

Quadro A.4 – Resultados dos testes dirigidos.

Nº	Caso de teste	Cenário	O que se verifica
1	Uso máximo de tokens (teste padrão)	Instância típica em que o orçamento pode ser preenchido	Valor ótimo e respeito ao limite W na seleção retornada
2	Corpus vazio	$N = 0$	Tabela de dimensão $1 \times (W + 1)$ com zeros; seleção vazia e valor 0, sem erro de execução
3	Tokens excedidos	Orçamento zerado ($W = 0$)	Nenhum artigo com custo positivo pode ser selecionado; resultado vazio
4	Borda exata	Artigo cujo custo é igual ao orçamento	Ausência de erro de índice (*off-by-one*) no fatiamento `custo:` e aceitação de custo = w (condição $\leq$, não $<$)
5	Artigo inviável	Custo do artigo superior ao orçamento ($c_i > W$)	Desvio por `continue`; o artigo nunca é selecionado
6	Tratamento de empates	Dois artigos com mesmo custo e mesmo valor	Resultado determinístico segundo o critério estrito com tolerância
7	Artigo sem relevância	Artigo com valor = 0	Artigo ignorado; nenhum token é gasto sem ganho
8	Alto orçamento de tokens	W elevado, seleção de grande quantidade de artigos	Escalabilidade: o tempo cresce conforme $O(N \cdot W)$, sem degradação anormal, graças à vetorização
9	Caso adversarial simples	Instância em que a estratégia gulosa falha (como a da Seção 5)	Refutação do método guloso: a programação dinâmica atinge o ótimo (7) e o guloso, não (5)
10	Consulta sem correspondência	Nenhum artigo com relevância para a consulta	Todos os valores nulos; seleção vazia e valor 0
11	Rejeição de IDs duplicados	Entrada com identificadores repetidos	`validar_entrada` rejeita a entrada, preservando a integridade da reconstrução, que identifica por ID

Fonte: elaboração própria (2026).